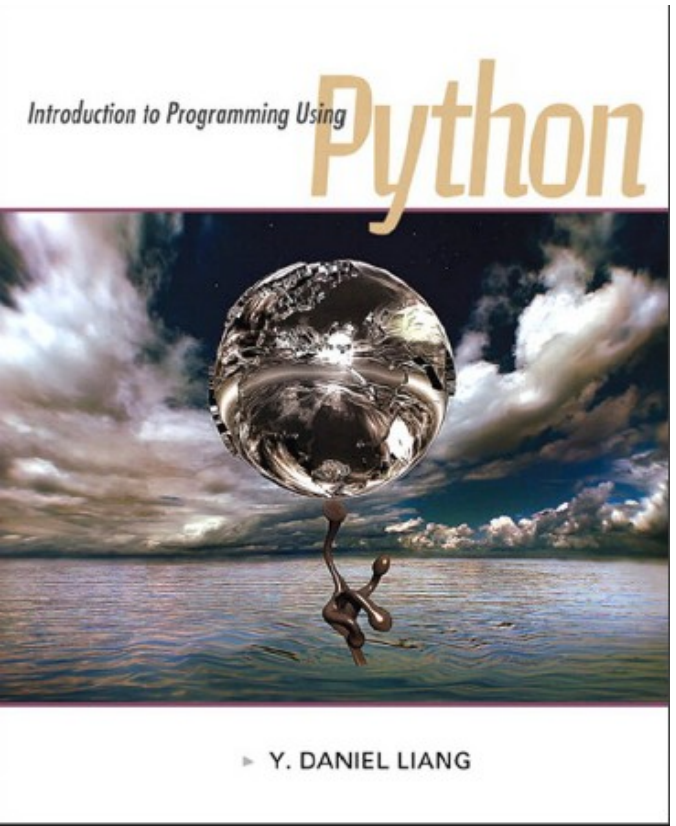


Chapter 05 Loops

Prepared By:

Eng. Eyad J. R. Elfarra

5.1 Introduction

A loop can be used to tell a program to execute statements repeatedly.

Motivation

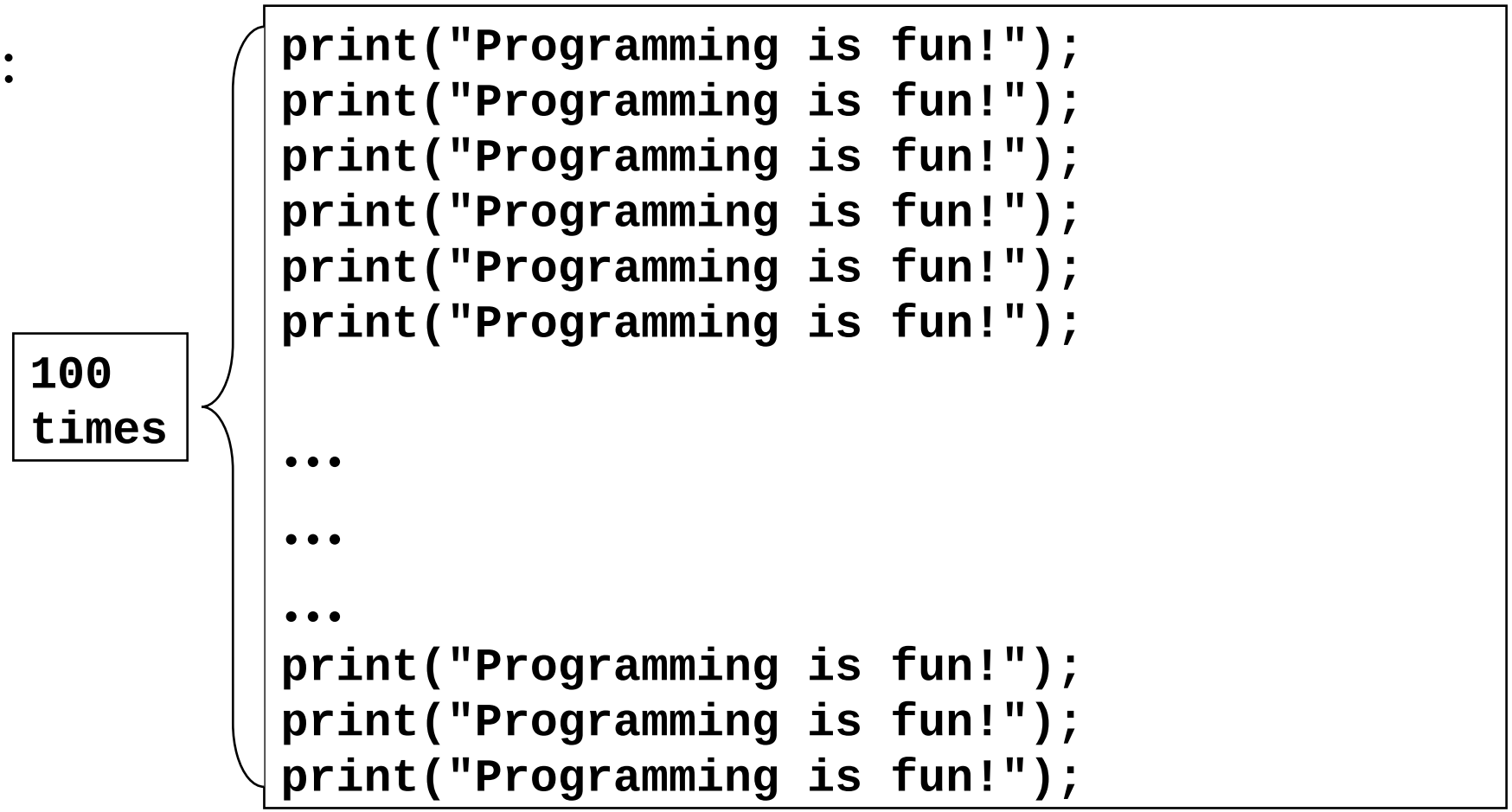
Suppose that you need to print a string (e.g., "Programming is fun!") a hundred times. It would be tedious to have to write the following statement a hundred times:

```
print("Programming is fun!");
```

So, how do you solve this problem?

Opening Problem

- Problem:



Introducing while Loops

- Python provides a powerful construct called a **loop.**
- **Loops:** controls how many **times** in succession an operation (or a sequence of operations) is performed.
- By using a loop statement, you **don’t have to code** the print statement a hundred times;
  - you simply tell the computer to display a string that number of times.

```
count = 0
while count < 100:
    print("Programming is fun!")
    count = count + 1
```

Objectives

- To write programs for executing statements repeatedly by using a **while** loop (§5.2).
- To develop loops following the loop design strategy (§§5.2.1-5.2.3).
- To control a loop with the user's confirmation (§5.2.4).
- To control a loop with a sentinel value (§5.2.5).
- To obtain a large amount of input from a file by using input redirection instead of typing from the keyboard (§5.2.6).
- To use **for** loops to implement counter-controlled loops (§5.3).
- To write nested loops (§5.4).
- To learn the techniques for minimizing numerical errors (§5.5).
- To learn loops from a variety of examples (**GCD**, **FutureTuition**, **MonteCarloSimulation**, **PrimeNumber**) (§§5.6, 5.8).
- To implement program control with **break** and **continue** (§5.7).

Loops Types

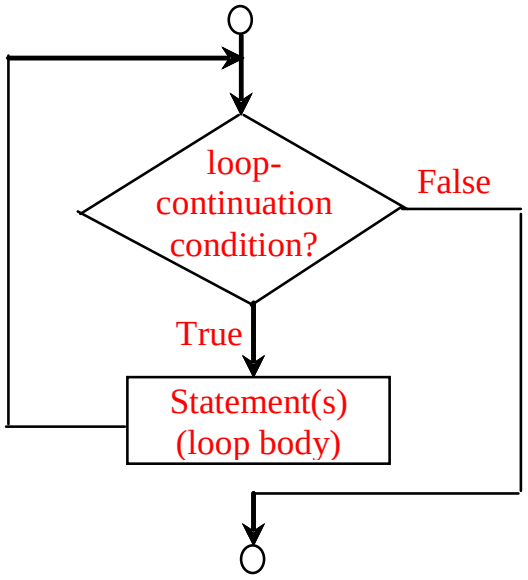
- Python provides two types of loop statements:
  - while loops and for loops.
  - The **while** loop is a **condition-controlled** loop; it is controlled by a true/false condition.
  - The **for** loop is a **count-controlled** loop that repeats a specified number of times.

5.2 The while Loop

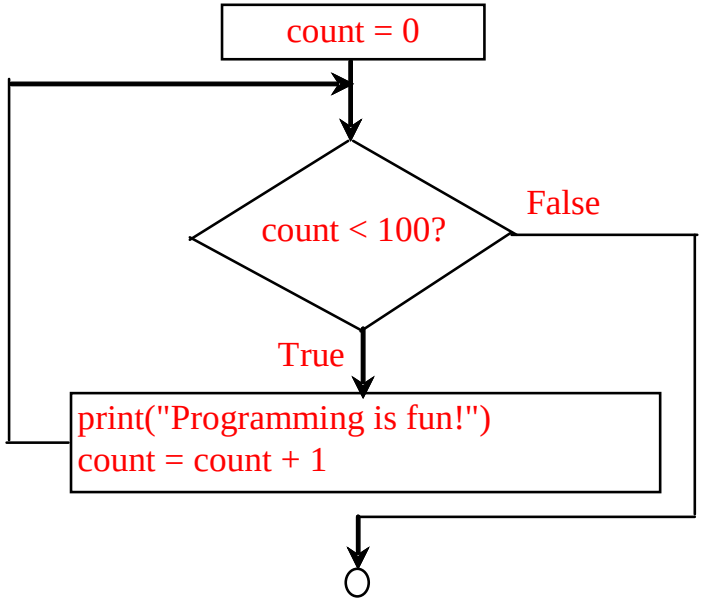
while Loop Flow Chart

```
while loop-continuation-condition:
      # Loop body
      Statement(s)
```

```
count = 0
while count < 100:
      print("Programming is fun!")
      count = count + 1
```



(a)



(b)

`while` Loop Flow Chart

- A **while** loop executes statements **repeatedly** as long as a condition remains true.
- A **single execution** of a loop body is called an **iteration** (or repetition) of the loop.
- Each loop contains a **loop-continuation-condition**, a Boolean expression that controls the body's execution.
  - It is evaluated each time to determine if the loop body should be executed.
  - If its evaluation is **True**, the loop body is **executed**; otherwise, the entire loop **terminates** and the program control turns to the statement that follows the while loop.

animation

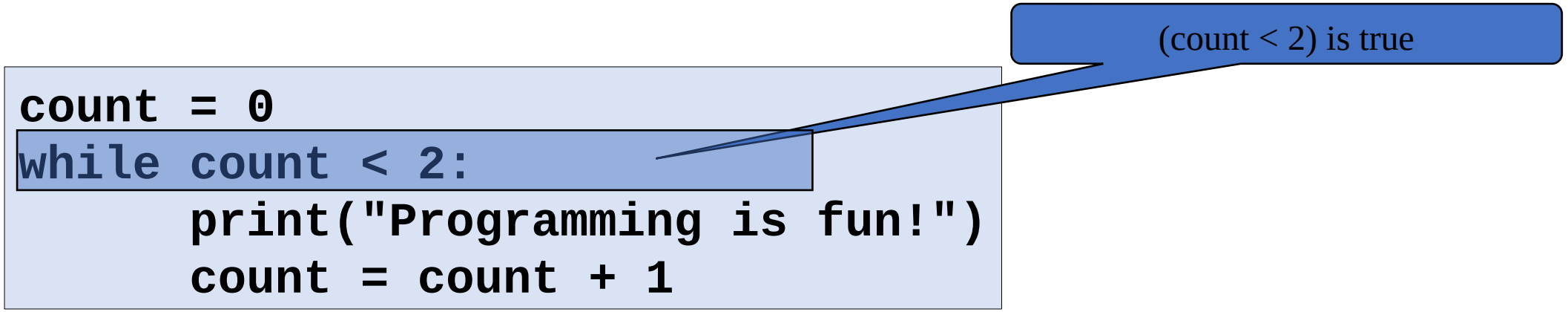
Trace while Loop

```
count = 0
while count < 2:
      print("Programming is fun!")
      count = count + 1
```

Initialize count

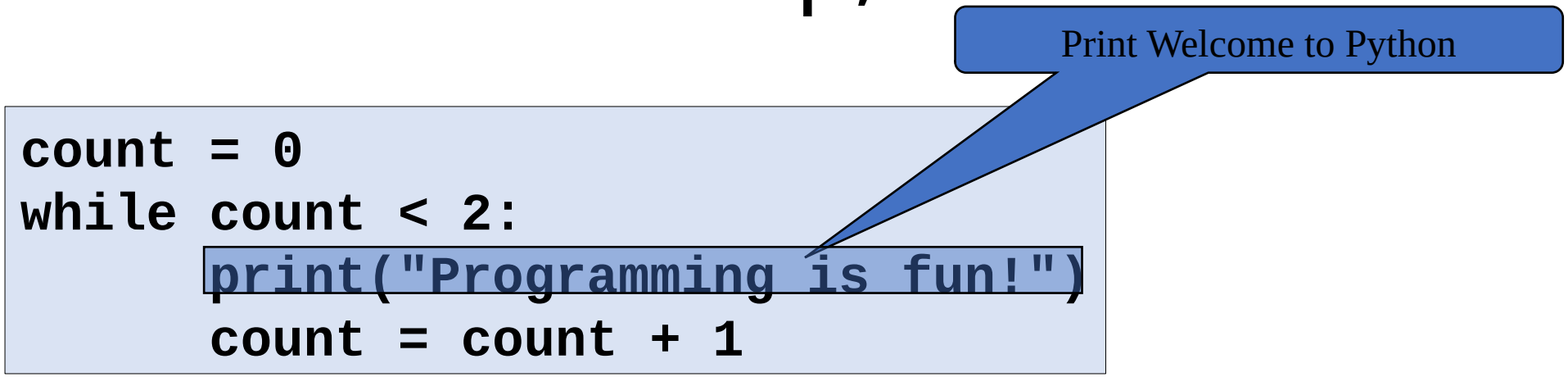
animation

Trace while Loop, cont.



animation

Trace while Loop, cont.



animation

Trace while Loop, cont.

```
count = 0
while count < 2:
      print("Programming is fun!")
      count = count + 1
```

Increase count by 1
count is 1 now

animation

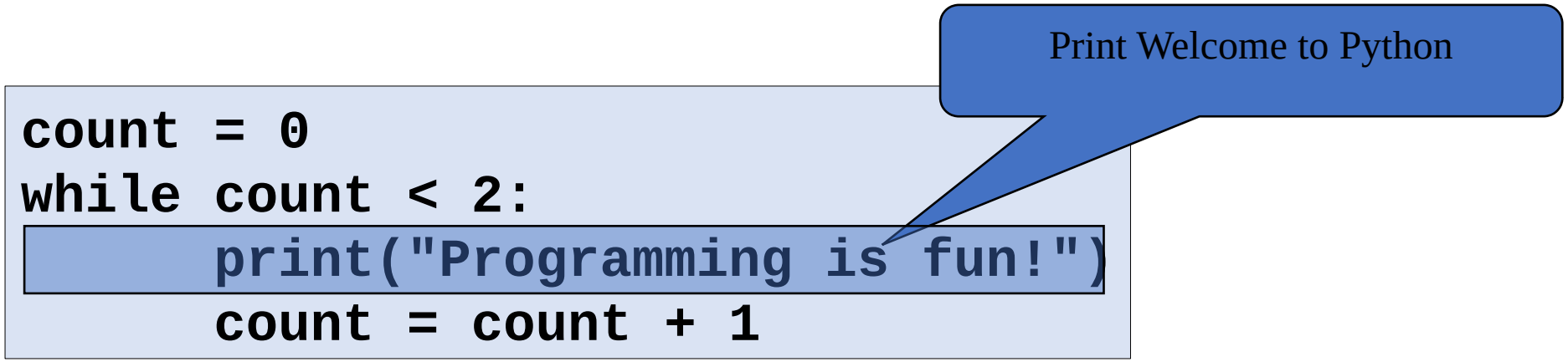
Trace while Loop, cont.

(count < 2) is still true since count is 1

```
count = 0
while count < 2:
        print("Programming is fun!")
        count = count + 1
```

animation

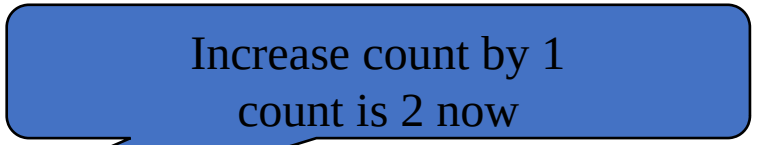
Trace while Loop, cont.



animation

Trace while Loop, cont.

```
count = 0
while count < 2:
        print("Programming is fun!")
        count = count + 1
```



animation

Trace while Loop, cont

(count < 2) is false since count is 2 now

```
count = 0
while count < 2:
        print("Programming is fun!")
        count = count + 1
```

animation

Trace while Loop

```
count = 0
while count < 2:
      print("Programming is fun!")
      count = count + 1
```



The loop exits. Execute the next statement after the loop.

Notes

- Note that the **entire loop body must be indented inside the loop**. Here the statement `i = i + 1` is not in the loop body.
- This loop is **infinite**, because **i is always 1** and **i < 10** will always be true.

```
sum = 0
i = 1
while i < 10:
    sum = sum + i
i = i + 1
```

Notes

- Make sure that the **loop-continuation-condition** eventually becomes **false** so that the loop will terminate.
- A common programming error involves infinite loops (i.e., the loop runs forever).
- If your program takes an **unusual long time** to run and does not stop, it may have an **infinite loop**.
- If you run the program from the command window, press **CTRL+C** to stop it.

Caution

- Programmers often mistakenly execute a loop one time more or less than intended.
- This kind of mistake is commonly known as the **off-by-one** error.
- For example, the following loop displays Programming is fun 101 times rather than 100 times.

```
count = 0
while count <= 100:
    print("Programming is fun!")
    count = count + 1
```

Problem: Guessing Numbers

- Write a program that **randomly generates an integer between 0 and 100**, inclusive. The program **prompts the user to enter a number continuously** until the number matches the randomly generated number. For **each user input**, the program tells the user whether the **input is too low or too high**, so the user can choose the next input intelligently.

```
Guess a magic number between 0 and 100
Enter your guess: 50 ↵Enter
Your guess is too high
Enter your guess: 25 ↵Enter
Your guess is too low
Enter your guess: 42 ↵Enter
Your guess is too high
Enter your guess: 39 ↵Enter
Yes, the number is 39
```

Without Loop

- **Step 1**: Generate a random number to be guessed

```
import random

# Generate a random number to be guessed
number = random.randint(0, 100)
```

- **Step 2**: Prompt the user to guess the number

```
print("Guess a magic number between 0 and 100")

# Prompt the user to guess the number
guess = eval(input("Enter your guess: "))
```

- **Step 3** : Check the Guess

```
if guess == number:
    print("Yes, the number is", number)
elif guess > number:
    print("Your guess is too high")
else:
    print("Your guess is too low")
```

With Loop

- **Step 4** : Using Loop

```
while guess != number:
    # Prompt the user to guess the number
    guess = eval(input("Enter your guess: "))

    if guess == number:
        print("Yes, the number is", number)
    elif guess > number:
        print("Your guess is too high")
    else:
        print("Your guess is too low")
```

Loop Design Strategies

- Writing a loop that works correctly is not an easy task for novice programmers.
- Three steps to write the loop:
  - **Step 1**: Identify the statements that need to be repeated.
  - **Step 2**: Wrap these statements in a loop like this:

```
while True:
    Statements
```

  - **Step 3**: Code the **loop-continuation-condition** and add appropriate statements for controlling the loop.

```
while loop-continuation-condition:
    Statements
    Additional statements for controlling the loop
```

Problem: An Advanced Math Learning Tool

- The Math subtraction learning tool program generates just one question for each run. You can use a loop to generate **questions repeatedly**. This example gives a program that generates five questions and reports the number of the correct answers after a student answers all five questions.

```
What is 1 - 1? 0 ↵Enter
You are correct!

What is 7 - 2? 5 ↵Enter
You are correct!

What is 9 - 3? 4 ↵Enter
Your answer is wrong.
9 - 3 is 6

What is 6 - 6? 0 ↵Enter
You are correct!

What is 9 - 6? 2 ↵Enter
Your answer is wrong.
9 - 6 is 3

Correct count is 3 out of 5
Test time is 10 seconds
```

SubtractionQuizLoop.py

Controlling a Loop with User Confirmation

- If you want the user to **decide** whether to **take another** iterations or not, you can offer for a **user confirmation**.

```
continueLoop = 'Y'
while continueLoop == 'Y':
    # Execute the loop body once
    ...

    # Prompt the user for confirmation
    continueLoop = input("Enter Y to continue and N to quit: ")
```

```
continueLoop = 'Y'

while continueLoop.upper() == 'Y':
    # Execute the loop body once
    print("Loop is running...")

    # Prompt the user for confirmation
    continueLoop = input("Enter Y to continue and N to quit: ")

print("Program ended.")
```

Ending a Loop with a Sentinel Value

Often the number of times a loop is executed is not predetermined. You may use an input value to signify the end of the loop. Such a value is known as a ***sentinel value***.

Write a program that reads and calculates the sum of an unspecified number of integers. **The input 0 signifies the end of the input.**

```
Enter an integer (the input ends if it is 0): 2 Enter
Enter an integer (the input ends if it is 0): 3 Enter
Enter an integer (the input ends if it is 0): 4 Enter
Enter an integer (the input ends if it is 0): 0 Enter
The sum is 9
```

SentinelValue.py

Caution

Don’t use **floating-point** values for **equality checking** in a loop control. Since floating-point values are approximations for some values, using them could result in imprecise counter values and inaccurate results. Consider the following code for computing 1 + 0.9 + 0.8 + ... + 0.1:

```
item = 1
sum = 0
while item != 0: # No guarantee item will be 0
    sum += item
    item -= 0.1
print(sum)
```

Variable item starts with 1 and is reduced by 0.1 every time the loop body is executed. The loop should terminate when item becomes 0. However, there is **no guarantee** that item will be **exactly 0**, because the floating-point arithmetic is approximated. This loop seems OK on the surface, but it is actually an infinite loop.

Input and Output Redirections

- **Problem**
  - Typing large amounts of input manually is inconvenient.
- **Input Redirection**
  - Reads input from a file instead of keyboard
  - python SentinelValue.py < input.txt

  - Program reads values automatically.
  - **Output**: sum = 9

input.txt

```
2
3
4
0
```

Input and Output Redirections

- **Output Redirection**
  - Sends output to a file instead of screen
  - python Script.py > output.txt
- **Both Together**
  - python SentinelValue.py < input.txt > output.txt
  - Input comes from file
  - Output saved to file

5.3 The for Loop

for Loop

- A Python for loop iterates through each value in a sequence.
- when you know exactly how many times the loop body needs to be executed.
  - A loop of this type is called a **counter-controlled** loop.
  - In general, the loop can be written as follows:

```
i = initialValue  # Initialize loop-control variable
while i < endValue:
    # Loop body
    ...
    i += 1  # Adjust loop-control variable
```

For Loop

- A for loop can be used to simplify the preceding loop:

```
for i in range(initialValue, endValue):
    # Loop body
```

- In general, the syntax of a for loop is:

```
for var in sequence:
    # Loop body
```

range(a, b)

```
>>> for v in range(4, 8):
...     print(v)
...
4
5
6
7
>>>
```

range(b)

```
>>> for i in range(4):
...     print(i)
...
0
1
2
3
>>>
```

range(a, b, step)

```
>>> for v in range(3, 9, 2):
...     print(v)
...
3
5
7
>>>
```

range(a, b, step)

```
>>> for v in range(5, 1, -1):
...     print(v)
...
5
4
3
2
>>>
```

Note

- The numbers in the range function must be integers.
- For example, range(1.5, 8.5), range(8.5), or range(1.5, 8.5, 1) would be wrong.

5.4 Nested Loops

A loop can be nested inside another loop.

Nested Loops

- Nested loops consist of an **outer loop** and **one or more inner loops**.
- Each time the outer loop is repeated, the inner loops are reentered and started anew.

MultiplicationTable.py

- Problem: Write a program that uses nested for loops to print a multiplication table.

```
                Multiplication Table
          1   2   3   4   5   6   7   8   9
    ------------------------------------------
1   |     1   2   3   4   5   6   7   8   9
2   |     2   4   6   8  10  12  14  16  18
3   |     3   6   9  12  15  18  21  24  27
4   |     4   8  12  16  20  24  28  32  36
5   |     5  10  15  20  25  30  35  40  45
6   |     6  12  18  24  30  36  42  48  54
7   |     7  14  21  28  35  42  49  56  63
8   |     8  16  24  32  40  48  56  64  72
9   |     9  18  27  36  45  54  63  72  81
```

MultiplicationTable.py

```
1  print("Multiplication Table")
2
3  # Display the header row
4  print("   |", end='')
5  for j in range(1, 10):
6      print(format(j, "4d"), end='')
7  print()
8
9  print("-" * 40)
10
11 # Display table body
12 for i in range(1, 10):
13     print(format(i, "2d"), "|", end='')
14     for j in range(1, 10):
15         print(format(i * j, "4d"), end='')
16     print()
```

Note

- Be aware that a nested loop may take a long time to run. Consider the following loop nested in three levels:

```
for i in range(1000):
    for j in range(1000):
        for k in range(1000):
            Perform an action
```

- The action is performed 1,000,000,000 times.
- If it takes **1 millisecond** to perform the action, the total time to run the loop would be more than **277 hours**.

5.5 Minimizing Numerical Errors

Using floating-point numbers in the loop-continuation-condition may cause numeric errors.

Minimizing Numerical Errors

- Numeric errors involving floating-point numbers are inevitable. This section discusses how to minimize such errors through an example.
- Here is an example that sums a series that starts with 0.01 and ends with 1.0. The numbers in the series will increment by 0.01, as follows: 0.01 + 0.02 + 0.03 and so on.

```
1  # Initialize sum
2  sum = 0
3
4  # Add 0.01, 0.02, ..., 0.99, 1 to sum
5  i = 0.01
6
7  while i <= 1.0:
8      sum += i
9      i = i + 0.01
10
11 # Display result
12 print("The sum is", sum)
```

Results

```
F:\IUG\fundamentals Programming\Codes\Ch 05>TestSum.py
The sum is 49.50000000000003
```

Expected Result

50.5

Why??

Why the Problem Happens

- **Floating-Point Representation**
  - Numbers like **0.01** cannot be stored exactly in binary.
  - Computers store them as approximations
  - Small rounding errors accumulate
- Example: **i** may become **0.999999** instead of **1.0**
  - Condition **i <= 1.0** may fail unexpectedly

Solution

- **Use an Integer Counter**
- Avoid floating-point comparison.
- Control loop using integers.
- Convert to decimal inside the loop.
- **Suggested Solutions**:
  - Using while Loop with Counter.
  - Using for Loop (Best Practice).

Using while Loop with Counter

```
# Initialize sum
total = 0
count = 0

while count < 100:
    total += (count + 1) / 100
    count += 1

print("The sum is", total)
```

- **Advantage:**
  - No precision issues
  - All values included

Using for Loop (Best Practice)

```
# Initialize sum
total = 0

for count in range(1, 101):
    total += count / 100

print("The sum is", total)
```

- **Why better?**
  - Cleaner and simpler
  - More readable
  - Less error-prone

5.6 Case Studies

Loops are fundamental in programming. The ability to write loops is essential in learning programming.

Problem:
Finding the Greatest Common Divisor

- Problem: Write a program that prompts the user to enter two positive integers and finds their greatest common divisor.
- Solution: Suppose you enter two integers 4 and 2, their greatest common divisor is 2. Suppose you enter two integers 16 and 24, their greatest common divisor is 8. So, how do you find the greatest common divisor? Let the two input integers be n1 and n2. You know number 1 is a common divisor, but it may not be the greatest commons divisor. So you can check whether k (for k = 2, 3, 4, and so on) is a common divisor for n1 and n2, until k is greater than n1 or n2.

GreatestCommonDivisor.py

```
# Prompt the user to enter two integers
n1 = eval(input("Enter first integer: "))
n2 = eval(input("Enter second integer: "))

gcd = 1
k = 2
while k <= n1 and k <= n2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1

print("The greatest common divisor for",
    n1, "and", n2, "is", gcd)
```

```
Enter first integer: 125 Enter
Enter second integer: 2525 Enter
The greatest common divisor for 125 and 2525 is 25
```

Problem: Predicting the Future Tuition

- Problem: Suppose that the tuition(الرسوم الدراسية) for a university is $10,000 this year and tuition increases 7% every year. In how many years will the tuition be doubled?

```
year = 0  # Year 0
tuition = 10000
year += 1 # Year 1
tuition = tuition * 1.07
year += 1 # Year 2
tuition = tuition * 1.07
year += 1 # Year 3
tuition = tuition * 1.07
```

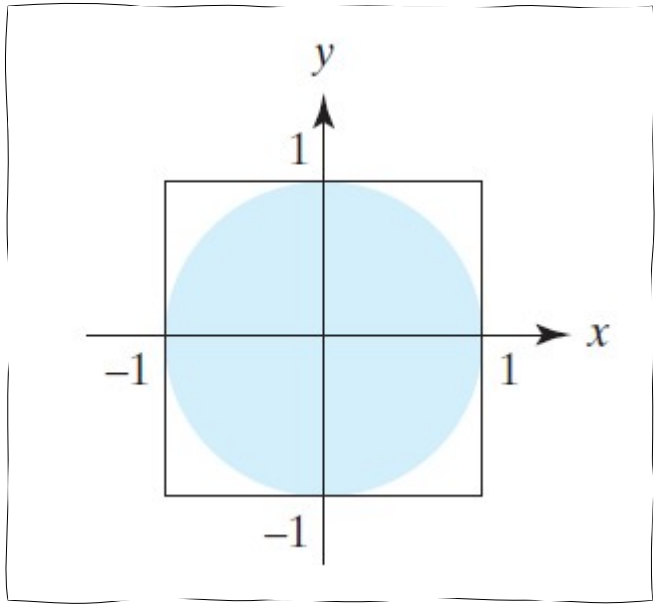
FutureTuition.py

```
1  year = 0   # Year 0
2  tuition = 10000   # Year 1
3
4  while tuition < 20000:
5      year += 1
6      tuition = tuition * 1.07
7
8  print("Tuition will be doubled in", year, "years")
9  print("Tuition will be $" + format(tuition, ".2f"), "in", year, "years")
```

```
Tuition will be doubled in 11 years
Tuition will be $21048.52 in 11 years
```

Problem: *Monte Carlo Simulation*

- The Monte Carlo simulation refers to a technique that uses random numbers and probability to solve problems. This method has a wide range of applications in computational mathematics, physics, chemistry, and finance. This section gives an example of using the Monto Carlo simulation for estimating π.



circleArea / squareArea = π / 4.

π can be approximated as 4 * numberOfHits / 1000000.

MonteCarloSimulation.py

```python
import random

NUMBER_OF_TRIALS = 1000000  # Constant, more trials more accurate PI
numberOfHits = 0

for i in range(NUMBER_OF_TRIALS):
    x = random.random() * 2 - 1  #* 2 - 1 converts it to range [-1, 1]
    y = random.random() * 2 - 1

    if x * x + y * y <= 1: # inside the circle
        numberOfHits += 1

pi = 4 * numberOfHits / NUMBER_OF_TRIALS

print("PI is", pi)
```

```
PI is 3.14124
```

5.7 Keywords break and continue

The break *and* continue *keywords provide additional controls to a loop.*

Breaking Out of a Loop

The **break** statement ends the current loop and jumps to the statement immediately following the loop

It is like a loop test that can happen anywhere in the body of the loop

```
while True:
    line = input('> ')
    if line == 'done' :
        break
    print(line)
print('Done!')
```

```
hello there

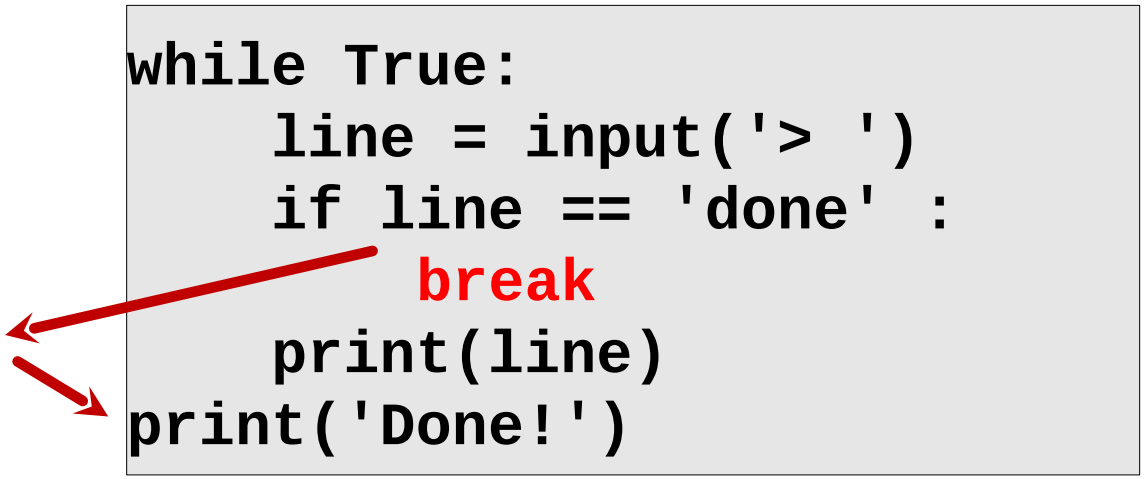
finished

done
```

Breaking Out of a Loop

The **break** statement ends the current loop and jumps to the statement immediately following the loop

It is like a loop test that can happen anywhere in the body of the loop



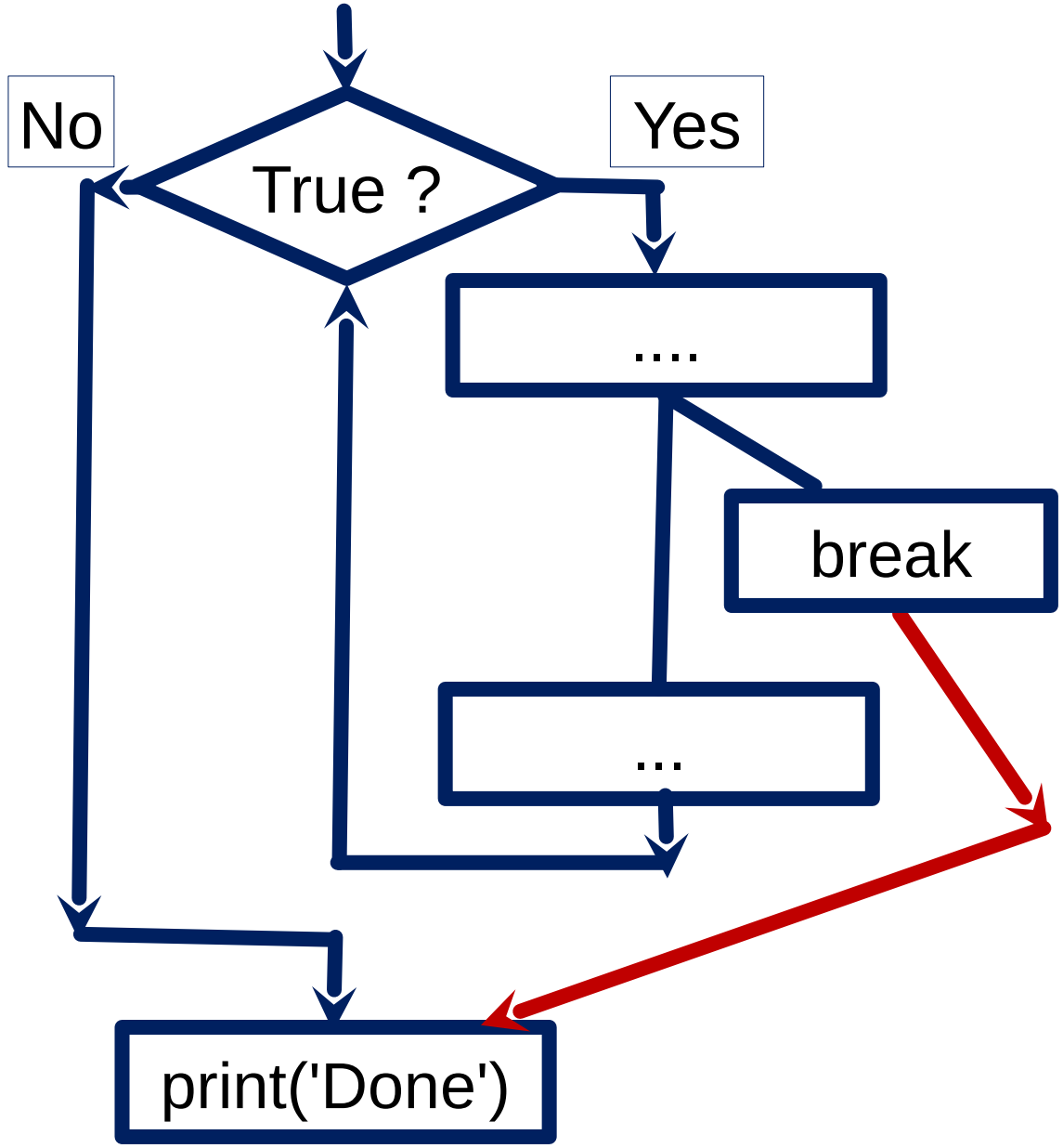
```
while True:
    line = input('> ')
    if line == 'done' :
        break
    print(line)
print('Done!')
```

```
hello there

finished

done
```

```
while True:
    line = input('> ')
    if line == 'done' :
        break
    print(line)
print('Done!')
```



Finishing an Iteration with continue

The continue statement ends the current iteration and jumps to the top of the loop and starts the next iteration

```
while True:
    line = input('> ')
    if line[0] == '#' :
        continue
    if line == 'done' :
        break
    print(line)
print('Done!')
```

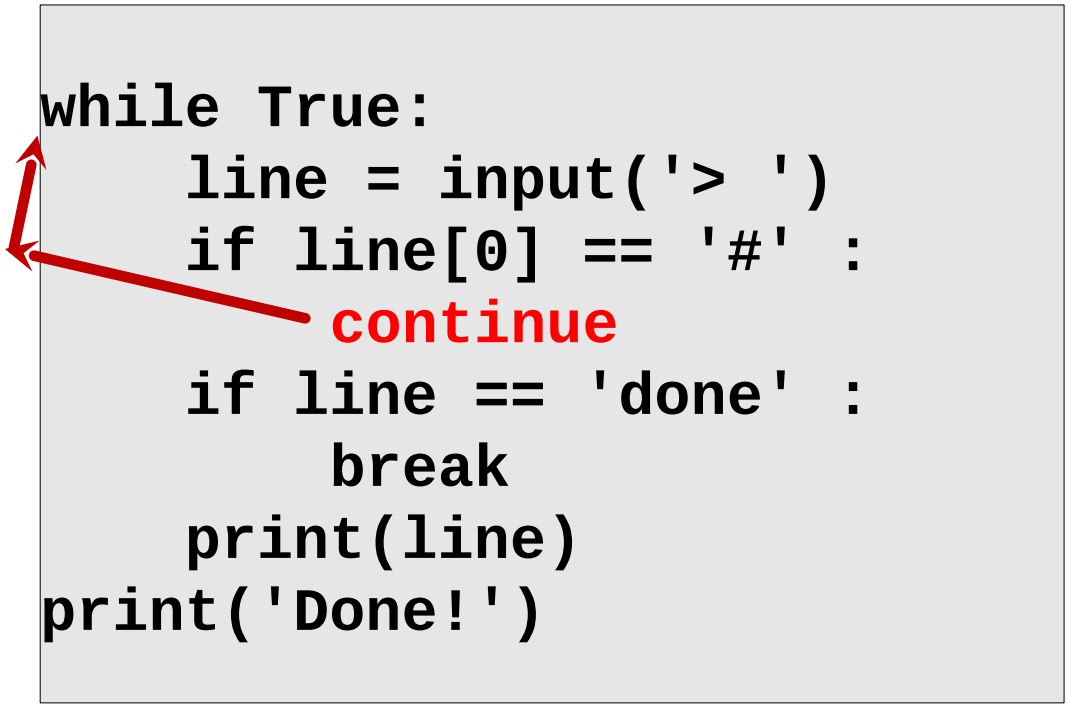
```
hello there

# don't print this
print this!

done
```

Finishing an Iteration with continue

The **continue** statement ends the **current iteration** and jumps to the **top of the loop** and starts the next iteration



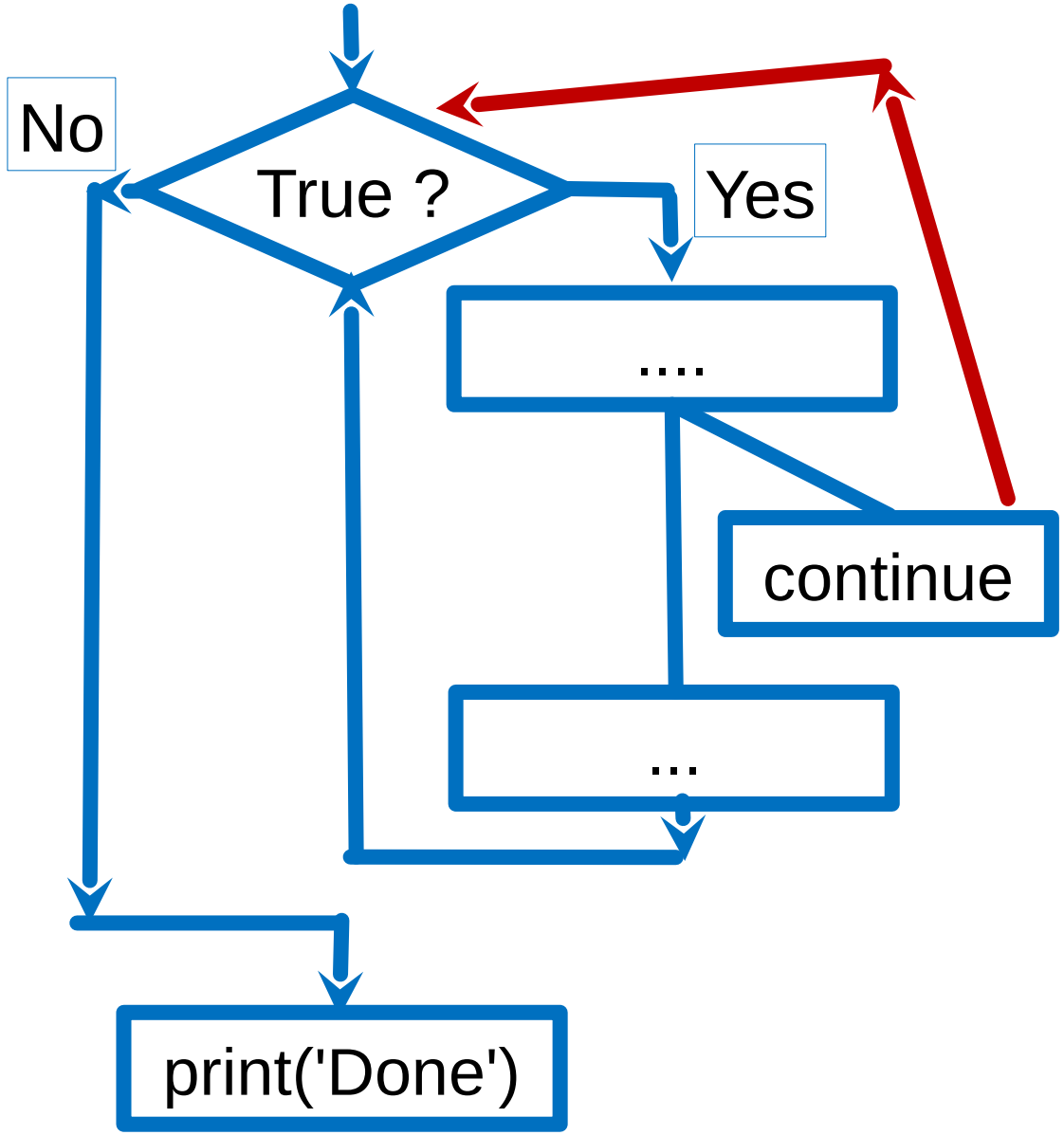
```
while True:
    line = input('> ')
    if line[0] == '#' :
        continue
    if line == 'done' :
        break
    print(line)
print('Done!')
```

hello there

don't print this
print this!

done

```
while True:
    line = raw_input('> ')
    if line[0] == '#' :
        continue
    if line == 'done' :
        break
    print(line)
print('Done!')
```



TestBreak.py

```
1      sum = 0
2      number = 0
3
4      while number < 20:
5          number += 1
6          sum += number
7          if sum >= 100:
8              break
9
10     print("The number is", number)
11     print("The sum is", sum)
```

```
The number is 14
The sum is 105
```

TestContinue.py

```
1  sum = 0
2  number = 0
3
4  while number < 20:
5      number += 1
6      if number == 10 or number == 11:
7          continue
8      sum += number
9
10 print("The sum is", sum)
```

```
The sum is 189
```

5.8 Case Study: Displaying Prime Numbers

Problem: Displaying Prime Numbers

- Problem: Write a program that displays the **first 50 prime** numbers in five lines, each of which contains 10 numbers. An integer greater than 1 is *prime* if its only positive divisor is 1 or itself. For example, 2, 3, 5, and 7 are prime numbers, but 4, 6, 8, and 9 are not.
- Solution: The problem can be broken into the following tasks:
  - For number = 2, 3, 4, 5, 6, …, test whether the number is prime.
  - Determine whether a given number is prime.
  - Count the prime numbers.
  - Print each prime number, and print 10 numbers per line.
  -

PrimeNumber.py

```
The first 50 prime numbers are

   2    3    5    7   11   13   17   19   23   29
  31   37   41   43   47   53   59   61   67   71
  73   79   83   89   97  101  103  107  109  113
 127  131  137  139  149  151  157  163  167  173
 179  181  191  193  197  199  211  223  227  229
```

What is the Largest Number?

3 41 12 9 74 15

What is the Largest Number?

largest_so_far -1

What is the Largest Number?

3

largest_so_far 3

What is the Largest Number?

41

largest_so_far

41

What is the Largest Number?

12

largest_so_far

41

What is the Largest Number?

9

largest_so_far 41

What is the Largest Number?

74

largest_so_far



What is the Largest Number?

15

largest_so_far



What is the Largest Number?

3 41 12 9 74 15

largest_so_far



Finding the Largest Value

```
largest_so_far = -1
print('Before', largest_so_far)
for the_num in [9, 41, 12, 3, 74, 15] :
   if the_num > largest_so_far :
      largest_so_far = the_num
   print(largest_so_far, the_num)

print('After', largest_so_far)
```

```
$ python largest.py
Before -1
9  9
41  41
41 12
41  3
74  74
74  15
After 74
```

variable

number we are looking at

largest value we have seen so far

largest value we have seen so far

End of Chapter 05